

SHIV NADAR UNIVERSITY CHENNAI

School of Engineering

CS3807 – Deep Learning Laboratory

Experiment – 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Name	: Srivatsan S
Register No.	: 24011101093
Degree & Branch	: B.Tech Artificial Intelligence & Data Science
Class / Section	: AIDS – B
Semester	: V
Subject Code	: CS3807
Subject Name	: Deep Learning Laboratory

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image pre-processing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

SOURCE CODE

<https://github.com/sriwhatsnext/dllab/tree/main/Week-02-MLP>

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

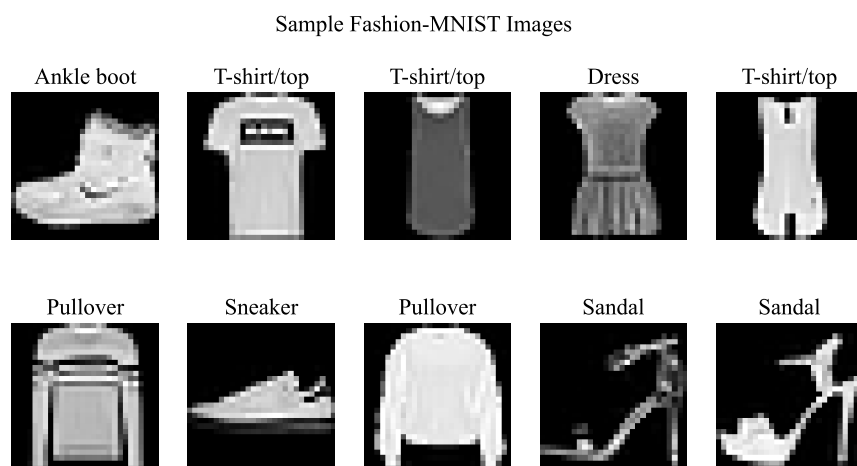
Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

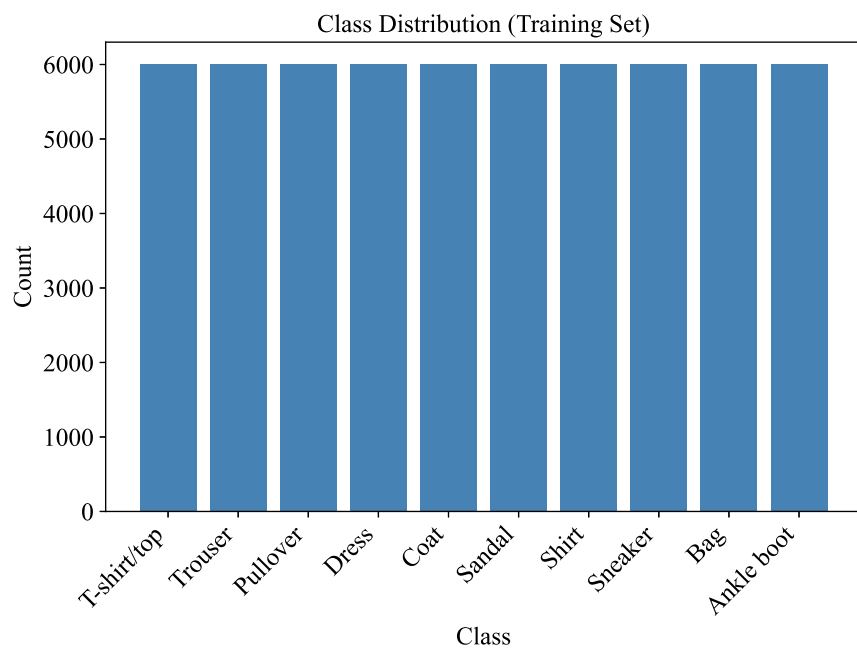
1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Generated Plots with Interpretation

8.1 Sample Images

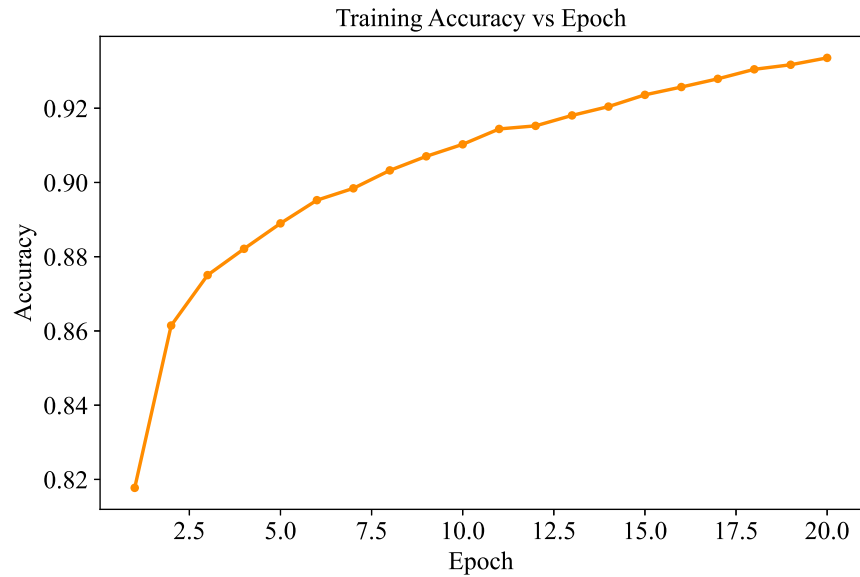


8.2 Class Distribution



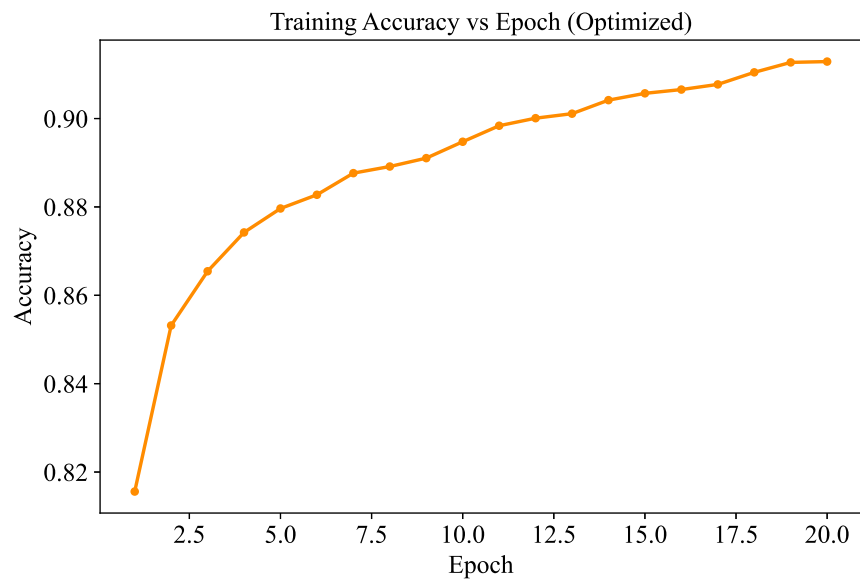
Interpretation: The database is balanced as each of the classes contains the same number of images.

8.3 Training Accuracy vs Epoch



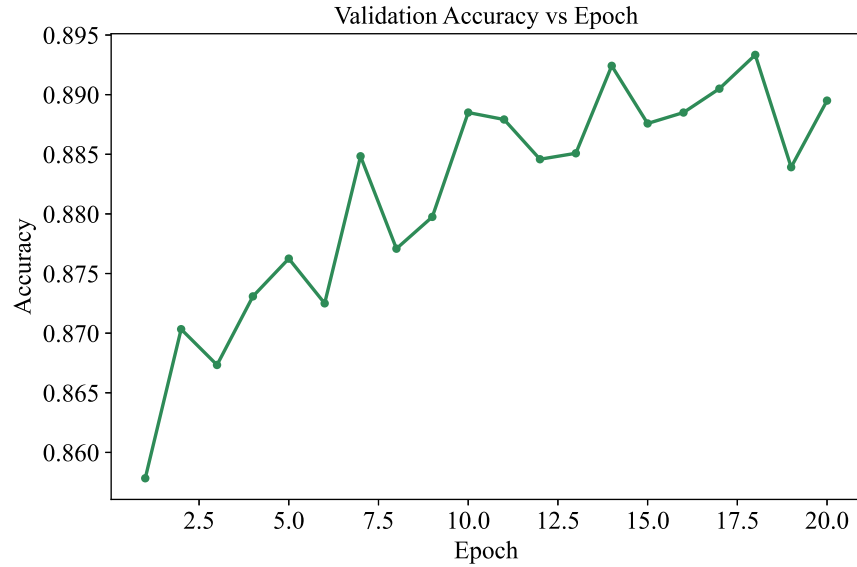
Interpretation: Training accuracy steadily rises and flattens out at the last fewer epochs.

Training Accuracy vs Epoch (Optimized)



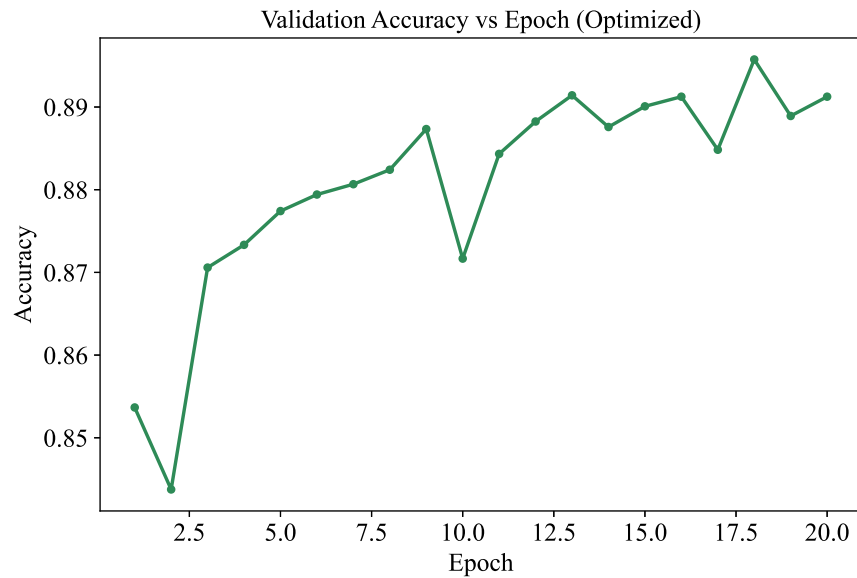
Interpretation: The optimized training accuracy follows a similar trend but still is lower than the baseline's value .

8.4 Validation Accuracy vs Epoch



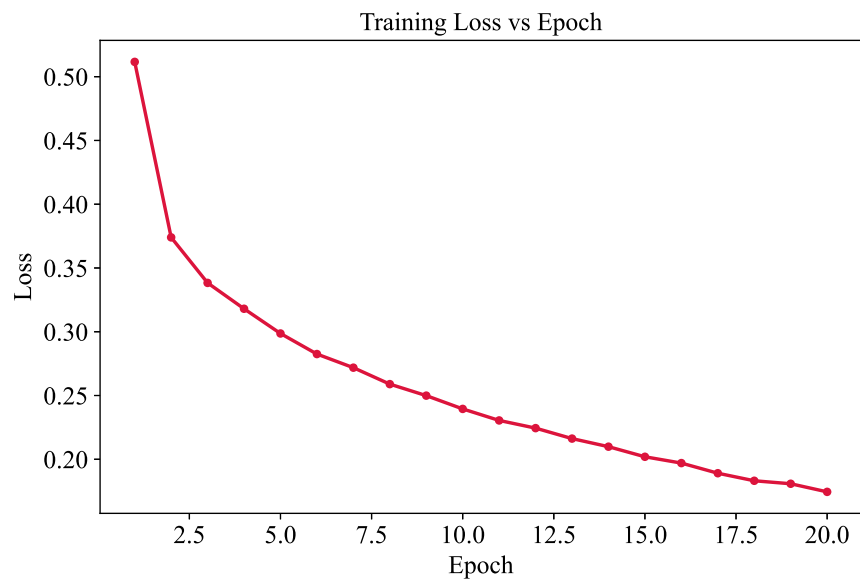
Interpretation: Validation accuracy fluctuates within the range 0.86 to 0.9 across the epochs

Validation Accuracy vs Epoch (Optimized)



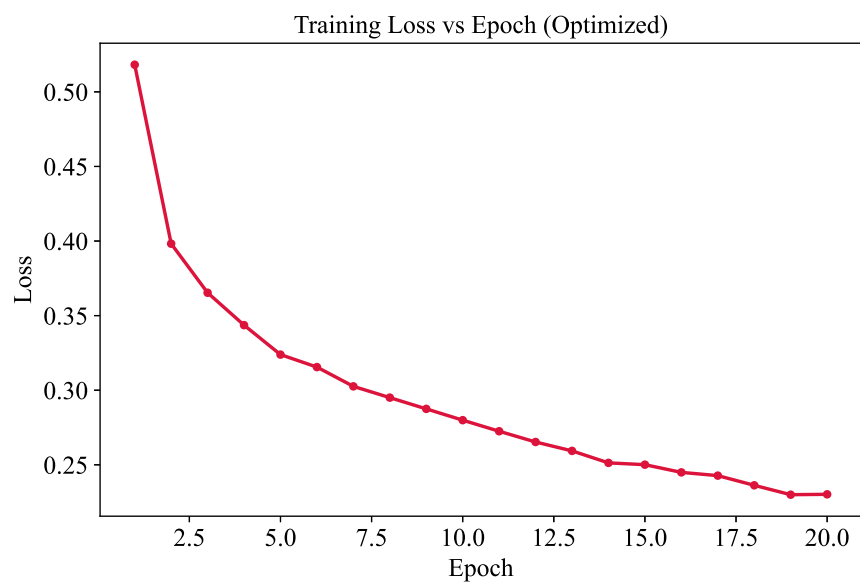
Interpretation: The validation accuracy of the optimized model has a higher peak than the baseline version and the gap between the validation and training accuracy for the optimized model is narrower than the baseline suggesting that the optimized model generalizes better

8.5 Training Loss vs Epoch



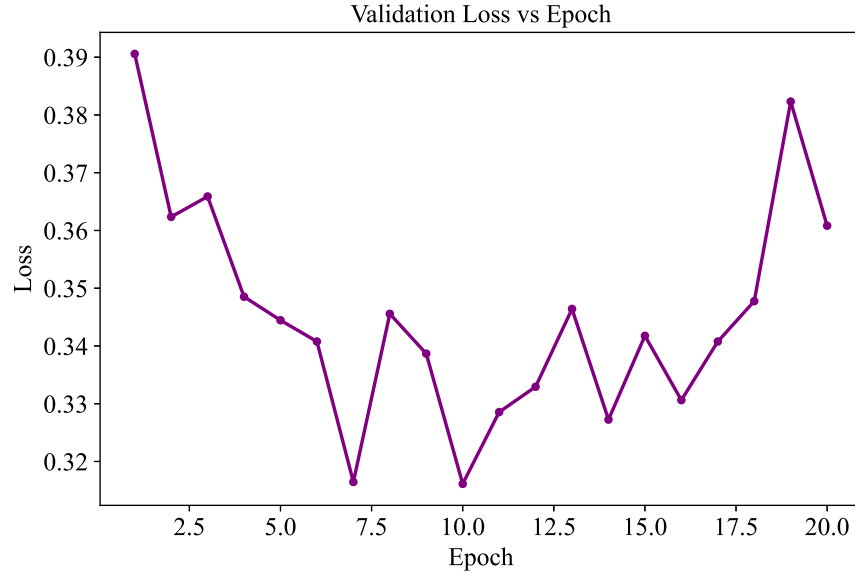
Interpretation: Across the epochs the training loss decreases smoothly with stable convergence of the adam optimizer .

Training Loss vs Epoch (Optimized)



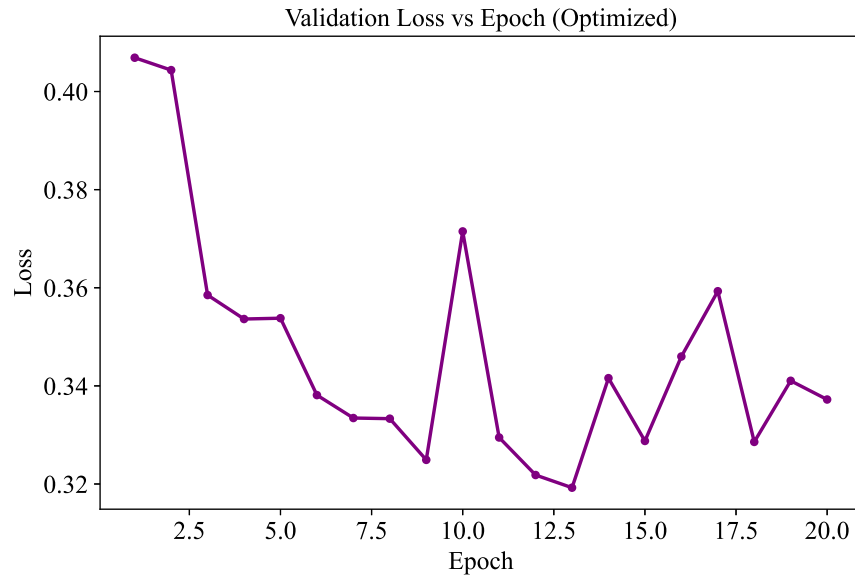
Interpretation: The training loss of the optimized model decreases almost similar to the baseline model

8.6 Validation Loss vs Epoch



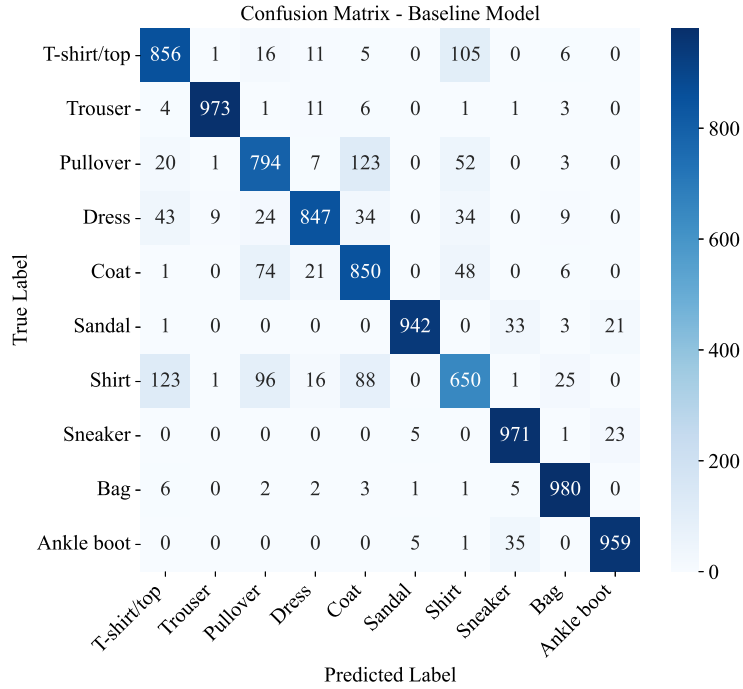
Interpretation: Around the range 7 to 11 the validation loss is at a valley and as the increased number of epochs due to the model overfitting causing the validation loss to increase .

Validation Loss vs Epoch (Optimized)



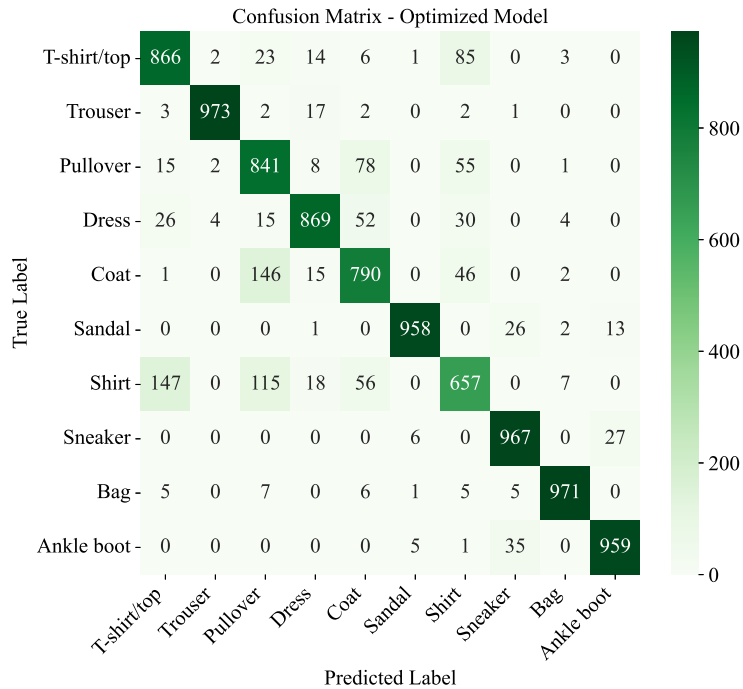
Interpretation: Validation loss of the optimized model is slightly better than that of the baseline model and also as the number of epochs increase the model does not show the overfitting nature as much as compared to the baseline model . This is because of the dropout regularization used in the optimized architecture.

8.7 Confusion Matrix (Baseline)



Interpretation: Diagonal elements being dark suggests that the model correctly classified the classes with fewer exceptions having more than 100 misclassified samples.

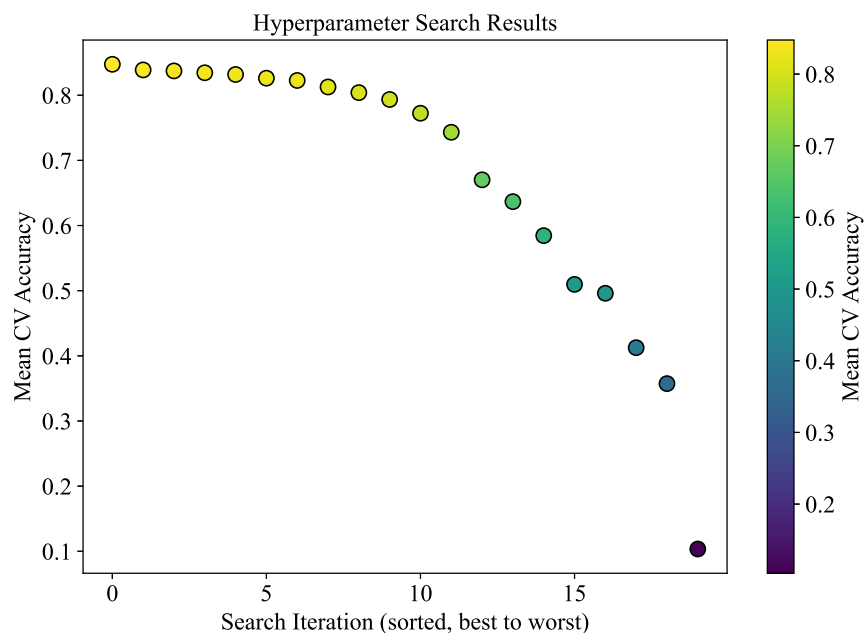
Confusion Matrix (Optimized)



Interpretation: While some classes's f1 scored has improve and some have decreased the scores of already lower scored classes doesnt show significant improvement in the optimized version

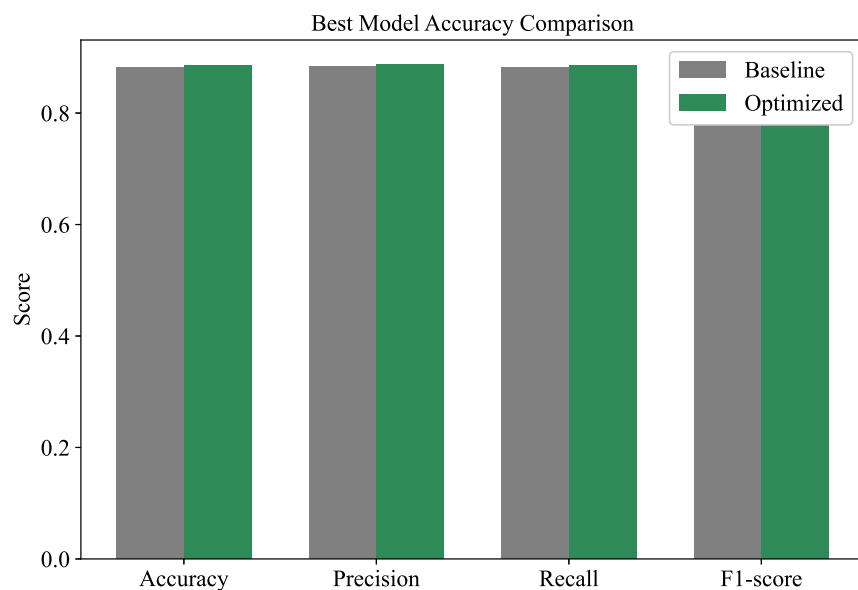
suggesting the limitations of optimization and the importance of the datapoints in the dataset

8.8 Hyperparameter Search Results



Interpretation:

8.9 Best Model Accuracy Comparison



Interpretation: The baseline and the optimized model do not show significant change in the f1-score suggesting the baseline architecture was good enough for this dataset and problem

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	ReLU
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8475
Testing Accuracy	0.8851

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8822	0.8851
Precision	0.8828	0.8867
Recall	0.8822	0.8851
F1-score	0.8817	0.8850
Training Time	—	206.93 s

10. Discussion

Answer the following:

1. **Which optimization method was used?**

Randomized Search (`RandomizedSearchCV`) with 5-fold cross-validation was used, sampling 20 candidate hyperparameter combinations from the defined search space, instead of exhaustively evaluating all possible combinations.

2. **Which hyperparameters were selected?**

hidden layer - 1, hidden neurons - 256, ReLU activation, Adam optimizer, learning rate - 0.001, batch size - 16, dropout rate - 0.2, trained for - 20 epochs. This configuration achieved a mean cross-validation accuracy of 0.8475.

3. **Did optimization improve performance?**

the optimized model reached 88.51% accuracy, 88.67% precision, 88.51% recall, and 88.50% F1-score, compared to the baseline's 88.22%, 88.28%, 88.22%, and 88.17% respectively – an improvement of roughly 0.3–0.4 percentage points across all four metrics, but the baseline architecture was already a good enough model for this dataset.

4. **Which hyperparameter had the greatest impact?**

Learning rate and Hidden layer count

5. **Compare Grid Search and Randomized Search.** Grid search is when we evaluate every possible combination in the search space which is not feasible computationally as the number of parameters increase. Whereas randomized search is applied on a fixed number of combinations.

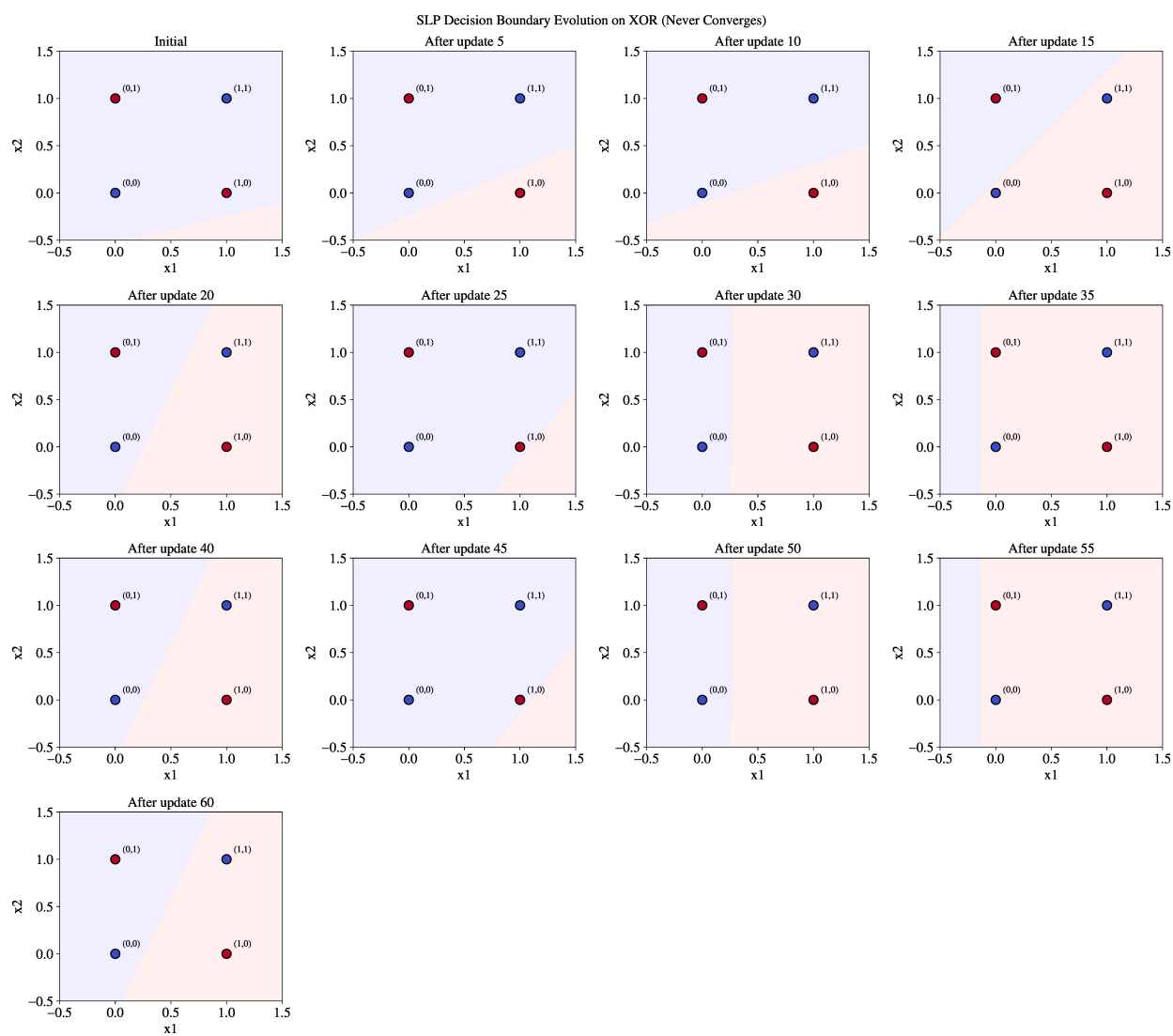
6. Recommend the best MLP configuration.

1 hidden layer of 256 neurons, ReLU activation, Adam optimizer, learning rate 0.001, batch size 16, dropout 0.2, trained for 20 epochs achieved scores accuracy (88.51%) and F1-score (88.50%) . The optimized model with dropout regularization giving a more stable validation curve i.e better generalization .

11. Additional Tasks

Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging.

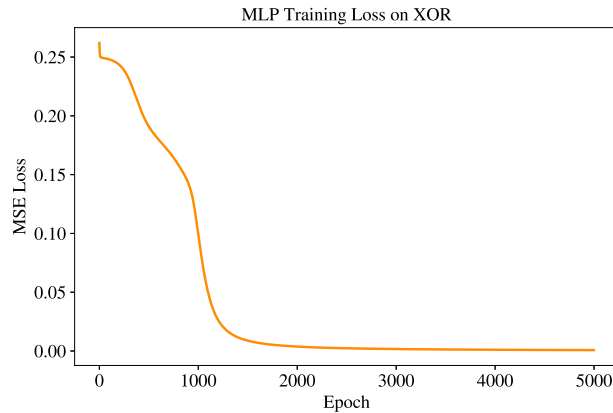
11.1 SLP XOR Decision boundary



Interpretation: Even after the end of 60 epochs the single layer perceptron could not learn

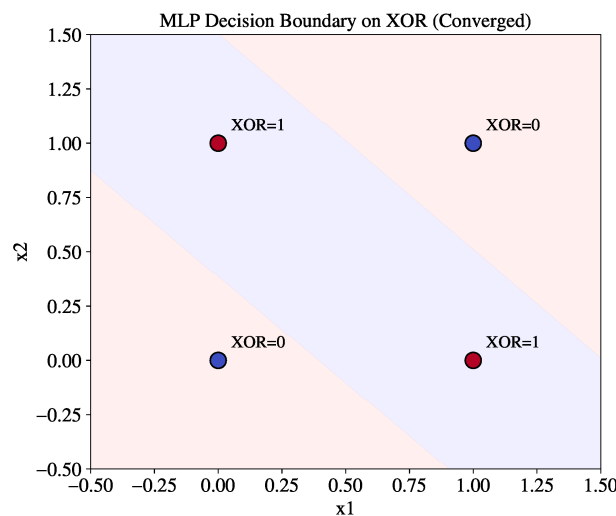
the xor problem suggesting that it can only be used to classify linearly separable data which XOR space is not.

11.3 MLP XOR loss curve



Interpretation: The training loss decreases significantly and flatlines at around 2000 epochs

11.4 MLP XOR Decision boundary



Interpretation: The decision boundary is non linear which could not have been learnt by a single perceptron hence justifies the usage of multilayer perceptrons .

11. Conclusion

Multi-layer perceptron was implemented and optimised with randomised search and dropout regularization . With the baseline model achieving 88.22 percent testing accuracy and the latter getting 88.51 which suggested that the baseline model was a good enough choice for this particular use case . Additionally the XOR problem was modelled by both the single layer and the multilayer perceptrons which proved that the single layer perceptron can only converge a linearly separable and a problem like xor can be solved by multi layer perceptron .

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.